

GOLANG

Living Textbook cho Software Engineering và DevOps/SRE

Edition nền móng

Được kiểm chứng với Go 1.27.1 - 20-09-2026

Markdown là nguồn gốc. PDF là bản đọc được, có thể tái tạo cục bộ.

Mục lục của edition này

Đây là edition nền móng. Mục lục tổng thể và thứ tự các phần tiếp theo được giữ trong `book/README.md` để có thể mở rộng mà không giả vờ rằng chúng đã được viết xong.

- Trước khi viết dòng Go đầu tiên
- Mở cửa vào Go
- Chương 1 — Đọc và viết một chương trình Go
- Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu
- Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi
- Chương 4 — Biên lỗi: để caller quyết định
- Chương 5 — Package là ranh giới

Trước khi viết dòng Go đầu tiên

Go không bắt đầu bằng câu hỏi “cú pháp nào còn thiếu?”. Nó bắt đầu bằng một vấn đề kỹ thuật rất cụ thể: đến năm 2007, máy tính đã nhanh hơn rất nhiều, nhưng việc xây và sửa một hệ thống phần mềm lớn vẫn bị kéo chậm bởi ngôn ngữ, build system và lượng bookkeeping mà lập trình viên phải mang theo.

Ở Google khi ấy, Robert Griesemer, Rob Pike và Ken Thompson làm việc giữa những codebase rất lớn, hệ thống nối mạng và phần cứng đang chuyển mạnh sang nhiều lõi. C++ và Java là lựa chọn phổ biến cho server; chúng đem lại những năng lực quan trọng, đồng thời tạo ra một lượng phức tạp đáng kể trong quá trình build, quản lý dependency và tổ chức chương trình. Ở phía khác, các ngôn ngữ động cho cảm giác phát triển nhanh hơn, nhưng đánh đổi một phần kiểm tra kiểu và đặc tính thực thi. Bài toán không phải là chọn một phe thắng phe còn lại. Bài toán là liệu một ngôn ngữ có thể đem hiệu quả biên dịch, hiệu năng và an toàn kiểu vào cùng một quy trình làm việc nhẹ hơn hay không.

Ngày 21 tháng 9 năm 2007, ba người bắt đầu phác thảo mục tiêu của một ngôn ngữ mới trên bảng trắng. Vài ngày sau, ngày 25 tháng 9, Pike đề xuất cái tên Go. Thiết kế ban đầu diễn ra song song với công việc khác; đầu năm 2008, Thompson đã có compiler thử nghiệm sinh mã C để thử các ý tưởng. Đến giữa năm, dự án đủ rõ để hướng tới compiler dùng trong thực tế. Ian Lance Taylor bắt đầu GCC front end vào tháng 5 năm 2008; Russ Cox gia nhập cuối năm ấy và giúp đưa prototype, library cùng tooling tiến gần một ngôn ngữ có thể dùng.



Hình 1 — Dòng thời gian không kể mọi release. Nó chỉ giữ các bước làm thay đổi lời hứa của Go với người viết phần mềm: công khai, ổn định, triển khai lại nền tảng, dependency rõ ràng và generics tương thích ngược.

Go được mở mã nguồn ngày 10 tháng 11 năm 2009. Ba năm đầu là giai đoạn ngôn ngữ còn thay đổi nhanh: nhóm thiết kế và cộng đồng thử, bỏ, sửa, rồi mới chốt một nền tảng ổn định. Go 1 phát hành ngày 28 tháng 3 năm 2012. Từ đây, Go 1 Compatibility Promise trở thành một phần của văn hóa dự án: code nguồn viết cho Go 1 được kỳ vọng tiếp tục build và chạy trong các bản Go 1 sau, với các ngoại lệ được nêu rõ. Lời hứa ấy làm thay đổi cách một feature được cân nhắc. Một bổ sung không chỉ cần hữu ích hôm nay; nó phải sống được cùng hệ sinh thái trong nhiều năm.

Go 1.5 là một cột mốc về implementation. Toolchain compiler chuyển từ C sang Go, runtime cũng trở thành Go kèm một phần assembly. Garbage collector được thiết kế lại để chạy phần lớn công việc đồng thời với chương trình; scheduler cho phép giá trị mặc định của GOMAXPROCS chuyển từ 1 sang số logical CPU. Đó là những thay đổi nằm dưới mặt ngôn ngữ, nhưng

chúng làm rõ mục tiêu ban đầu: một chương trình Go cần hợp với phần cứng nhiều lỗi, không bắt người viết phải tự gánh mọi chi tiết của runtime.

Những thay đổi lớn sau đó cũng đi theo nhịp tiến hóa thay vì đập đi làm lại. Go 1.11 đưa modules vào như một lựa chọn thay cho GOPATH, với versioning và package distribution tích hợp vào toolchain; mô hình này dần thành cách quản lý dependency quen thuộc. Go 1.18, phát hành năm 2022, đưa type parameters vào ngôn ngữ. Đến Go 1.27 năm 2026, generic methods nổi phần còn thiếu vào câu chuyện đó: method declaration có thể có type parameters riêng. “Go 2” vì thế không thành một nhánh rewrite tách rời; nhiều ý tưởng được thử, thảo luận và đưa dần vào dòng Go 1.x khi chúng minh được giá trị của chúng.

Một dòng họ ý tưởng, không phải một cây sao chép

Go thuộc nhiều dòng tư tưởng cùng lúc. Cú pháp cơ bản đặt nó gần họ C. Cách nhìn về declaration, package và sự rõ ràng cấu trúc nhận ảnh hưởng đáng kể từ Pascal, Modula và Oberon. Concurrency có một lịch sử dài hơn: CSP của Tony Hoare đi qua Newsqueak, Alef và Limbo trước khi channels trở thành một phần của Go. Những đường này giải thích vì sao một số ý tưởng nghe quen, chứ không làm Go thành bản sao của bất kỳ ngôn ngữ nào trong số đó.



Hình 2 — Nhánh CSP giải thích lịch sử của channels; nhánh C và Pascal/Modula/Oberon giải thích những điểm xuất phát khác. Thiết kế cuối cùng vẫn được tạo cho bài toán engineering mà Go nhắm đến.

Một ngôn ngữ được sinh ra cho codebase lớn không chỉ cần syntax gọn. Nó cần một cách làm việc chung quanh syntax: build nhanh, formatter, package/dependency rõ, compiler cho phản hồi sớm và tooling có thể phân

tích source. Đó là lý do Go thường được nói đến cùng `gofmt`, `go test` và `go command`, thay vì chỉ cùng một danh sách tính năng ngôn ngữ.

Thiết kế bằng cách từ chối

Trong giai đoạn đầu, nhóm sáng lập đặt nặng đồng thuận: một feature chỉ đi tiếp khi cả nhóm thấy nó thực sự cần. Câu hỏi không dừng ở “có thể thêm gì?”, mà còn là “có thể bỏ gì để hệ thống vẫn đủ sức làm việc?”. Kết quả là một bộ trade-off có chủ ý.

Một điều đáng nhớ: sự vắng mặt của inheritance hierarchy, operator overloading, exception-style error handling hay generics trong giai đoạn đầu không phải bản tuyên ngôn rằng các cơ chế ấy luôn tệ. Mỗi cơ chế có ích trong những ngữ cảnh khác. Go ưu tiên một mô hình dễ đọc, dễ phân tích, biên dịch nhanh và dễ giữ ổn định; cái giá là một số biểu đạt quen thuộc không có mặt hoặc đến muộn hơn.

Không có type hierarchy buộc người đọc đi tìm cây kế thừa; thay vào đó, interface và composition sẽ được nói kỹ khi ta đã đọc vững code cơ bản. Không có operator overloading giúp ý nghĩa của các toán tử ít thay đổi theo type. Error được trả như value thay vì theo đường exception-style. Và generics được hoãn lại cho đến khi thiết kế có thể đem giá trị đủ lớn so với độ phức tạp nó thêm vào type system và toolchain. Đây là các lựa chọn kỹ thuật có thể tranh luận, không phải luật đạo đức.

Nhìn lịch sử theo cách này giúp giải thích cảm giác đầu tiên khi đọc Go. Bạn sẽ thấy nhiều câu ngắn, ít keyword, declaration rõ và tooling hiện diện sớm. Những đặc điểm ấy là dấu vết của một mục tiêu: giảm ma sát cho người đang cùng người khác xây, đọc và sửa phần mềm lâu dài.

Sau gần hai thập kỷ tiến hóa, cách tốt nhất để gặp các quyết định ấy không phải đọc thêm lịch sử. Hãy đặt chúng trước mắt trong một tệp `.go` rất nhỏ.

Ghi chú kiểm chứng

1. Go Team. Frequently Asked Questions: Origins and Design. go.dev/doc/faq
2. Cox, Russ. Toward Go 2. 13 July 2017. go.dev/blog/toward-go2

3. Go Team. Go 1 and the Future of Go Programs. March 2012. go.dev/doc/go1compat
4. Gerrand, Andrew. Go 1.5 is released. 19 August 2015. go.dev/blog/go1.5
5. Bonventre, Andrew. Go 1.11 is released. 24 August 2018. go.dev/blog/go1.11
6. Go Team. Go 1.18 is released! 15 March 2022. go.dev/blog/go1.18
7. Go Team. Go 1.27 Release Notes. 2026. go.dev/doc/go1.27

Mở cửa vào Go

Bạn không cần thuộc ngay danh sách keyword để bắt đầu. Điều cần trước tiên là nhìn một tệp Go và biết: phần nào đặt tên cho chương trình, phần nào tạo giá trị, phần nào quyết định đường đi, và phần nào chỉ là một lời gọi hàm.

Cuốn sách này không đi bằng các mẫu bài học quá ngắn. Nó đi bằng những lần thay đổi cách nhìn: đọc được code trước, nhìn được giá trị chạy qua code sau, rồi mới đến những lúc nhiều goroutine, mạng, dữ liệu và production khiến trực giác ban đầu không còn đủ. Khi một chủ đề cần một lỗi, một trace hay một thí nghiệm để thấy rõ, chương đó sẽ bắt đầu từ bằng chứng ấy.

Cách đọc: đừng lướt qua code block. Mỗi lần gặp một đoạn ngắn, hãy thử nói thành lời nó tạo giá trị gì, giá trị đó được đặt tên ở đâu và điều gì sẽ xảy ra nếu đổi một token. Compiler là phản hồi nhanh nhất cho dự đoán của bạn.

Chương đầu không có mục tiêu “biết hết Go”. Mục tiêu nhỏ hơn, nhưng thiết yếu: bạn có thể đọc một chương trình console ngắn từ trên xuống, tự sửa nó, và hiểu vì sao một thay đổi là hợp lệ hoặc không hợp lệ.

Chương 1 — Đọc và viết một chương trình Go

Một tệp nguồn có hình học riêng: phần đầu đặt nó vào package, vài declaration đứng ở cấp tệp, rồi các block lồng vào nhau. Ta sẽ bắt đầu từ một chương trình rất nhỏ: nó nhận một mã trạng thái và in ra một nhãn mà người trực vận hành có thể đọc. Ví dụ đủ thực để có dữ liệu, lựa chọn và thông điệp, nhưng chưa che mất ngôn ngữ dưới một dự án.

```
package main

import "fmt"

const service = "checkout"

func classify(status int) string {
    if status >= 500 {
        return "failed"
    }
    return "healthy"
}

func main() {
    status := 503
    label := classify(status)
    fmt.Println(service, label)
}
```



Hình 3 — Hãy nhìn figure này như bản đồ định vị trước khi đọc chi tiết. Nó cho thấy package name, main package, function declaration và entry function là các khái niệm có quan hệ, nhưng không phải là một khái niệm duy nhất.

Chưa cần đọc từng dòng. Hãy chỉ theo ba câu hỏi: chương trình bắt đầu ở đâu, giá trị nào được tạo, và giá trị nào bị gửi ra màn hình? Câu trả lời là main, literal 503 cùng các giá trị trả về từ classify, và lời gọi Println.

Bản đồ của một tệp nguồn

Từ đầu xuống cuối, tệp trên có bốn vùng: package nói tệp thuộc chương trình nào; import cho phép dùng tên từ package khác; khai báo ở cấp tệp tạo const và hàm; cuối cùng, thân hàm main là nơi một chuỗi câu lệnh được chạy.

Dòng hoặc mảnh code	Nó là gì khi đọc	Nó có tác dụng gì
<code>package main</code>	package declaration	Đặt tệp vào package có tên main. Một executable hoàn chỉnh cần main package và func main không argument, không result.
<code>import "fmt"</code>	import declaration	Cho phép tham chiếu fmt từ standard library.
<code>const service = "checkout"</code>	declaration + literal	Gắn một tên không đổi với chuỗi.
<code>func classify(status int) string</code>	function declaration + signature	Đặt tên hàm, tham số kiểu int và kết quả kiểu string.
<code>status >= 500</code>	expression	Tính ra một giá trị bool.
<code>return "failed"</code>	statement	Kết thúc lời gọi hiện tại với một string.
<code>status := 503</code>	short variable declaration	Tạo biến status trong block main.
<code>fmt.Println(...)</code>	function call statement	Gọi hàm để có hiệu ứng in ra stdout.

Các nhãn này cho ta một cách chỉ vị trí khi compiler báo lỗi: lỗi có thể nằm trong một biểu thức, kiểu của một lời gọi, hay phạm vi của một tên thay vì chỉ là cảm giác mơ hồ rằng “code bị sai”.



Hình 4 — Đây là mô hình đọc, không phải sơ đồ nội bộ chính xác của Go compiler. Nó giải thích vì sao một dấu ngoặc, một tên không tồn tại và phép cộng sai kiểu thường bị bắt trước khi chương trình có cơ hội chạy.

Một tên chỉ có nghĩa trong phạm vi của nó

Ở dòng `const service`, `service` là identifier: một cái tên do người viết chọn. `fmt`, `Println`, `classify`, `status` và `label` cũng là identifier. Ngược lại, `package`, `import`, `const`, `func`, `if` và `return` là keyword; chúng dành riêng cho cấu trúc ngôn ngữ nên bạn không thể dùng chúng để tự đặt tên.

Tên không tự mang giá trị. Câu `status := 503` tạo một biến, rồi gán giá trị literal 503 vào biến đó. Trong cùng block, câu `label := classify(status)` tạo một biến khác, nhận kết quả mà hàm trả về. Tên tồn tại từ điểm khai báo đến cuối block bao quanh.

Phóng to một statement

Compiler không đọc `status := 503` như một câu tiếng Anh. Nó tách câu này thành các token có vai trò cú pháp, rồi ghép các token thành statement.

<code>status</code>	<code>:=</code>	<code>503</code>	
identifier	operator	integer	literal

Đây là một mức đọc khác với mức file hoặc function: identifier gọi lại một tên trong source, `:=` là operator khai báo-ngắn, còn `503` là integer literal. Khi nhìn code theo từng mức như vậy, một compiler message có chỗ để bám vào.

```
func main() {  
    status := 503  
    if status >= 500 {  
        label := "failed"  
        fmt.Println(label)  
    }  
    // fmt.Println(label) // không hợp lệ: label đã hết scope  
}
```

Khối `if` được tạo bởi cặp ngoặc nhọn. `label` chỉ hữu ích bên trong khối đó. Đây là một giới hạn có chủ ý: người đọc không phải lục lại toàn bộ tệp để đoán một tên có còn sống hay không.



Hình 5 — Scope không mô tả dữ liệu được lưu ở đâu trong bộ nhớ. Nó chỉ giới hạn nơi tên được phép được tham chiếu trong source.

Dừng lại để dự đoán: nếu bỏ comment ở dòng cuối, compiler phàn nàn về giá trị hay về tên? Câu trả lời là tên: `label` không còn được khai báo trong scope mà lời gọi `Println` đang đứng.

Làm chương trình lớn lên, mỗi lần một ý

Đừng viết đầy đủ từ đầu. Ta quan sát cùng một ý định khi nó nhận thêm một yêu cầu. Mỗi phiên bản giữ lại phần cũ có ích và chỉ thêm một khái niệm.

Literal: một kết quả cố định

```
package main

import "fmt"

func main() {
    fmt.Println("checkout healthy")
}
```

"checkout healthy" là string literal. Chương trình có một statement gọi hàm và không có quyết định nào. Nó đúng nếu thông điệp không bao giờ thay đổi.

Variable: dữ liệu thay đổi được đặt tên

```
func main() {
    service := "checkout"
    label := "healthy"
    fmt.Println(service, label)
}
```

Hãy nhìn một thay đổi trước khi học rule. Lần đầu, ta tạo status và cho nó giá trị. Lần sau, status đã có trong cùng block; ta chỉ thay giá trị của nó.

```
status := 503
status = 200
```

Vì dòng thứ hai không tạo một tên mới, nó dùng = thay vì :=. Short declaration chỉ dùng trong function body. Với nhiều tên ở vế trái, tên cũ có thể xuất hiện lại khi chúng đã có trong cùng block (hoặc parameter list tương ứng), type không đổi, và có ít nhất một non-blank variable mới. _ không được tính là tên mới. Đây là trường hợp tiện lợi khi bổ sung một kết quả mới mà vẫn giữ một biến đã có.

Muốn làm gì?	Viết gì?	Điều compiler cần biết
Tạo một biến mới và suy ra kiểu	<code>name := value</code>	Ít nhất một tên ở vế trái là mới trong scope hiện tại.
Gán lại biến đã có	<code>name = value</code>	Tên đã tồn tại và giá trị mới phù hợp kiểu.
Đặt giá trị không đổi ở cấp tệp	<code>const name = value</code>	Giá trị phải là hằng; không dùng <code>:=</code> .
Khai báo rõ kiểu trước	<code>var name Type = value</code>	Dùng khi kiểu hoặc zero value cần được nói rõ.

Expression: để chương trình tự kết luận

Literal 503 không tự nói rằng service thất bại. Ta cần một biểu thức tạo ra quyết định. `status >= 500` đọc là “status lớn hơn hoặc bằng 500”, và kết quả của nó là bool: true hoặc false.

```
func main() {
    status := 503
    if status >= 500 {
        fmt.Println("checkout failed")
    }
}
```

if nhận một biểu thức bool. Khi biểu thức là true, block của if được thực hiện; khi false, nó bị bỏ qua. Dấu ngoặc không chỉ để trang trí: chúng xác định chính xác nhóm câu lệnh phụ thuộc vào điều kiện.

Toán tử có thứ tự ưu tiên. Viết ngoặc khi chúng làm ý định rõ hơn thay vì bắt người đọc nhớ luật ưu tiên:

```
urgent := status >= 500 ||
    (status >= 400 && service == "checkout")
```

`&&` nghĩa là cả hai vế phải đúng; `||` nghĩa là ít nhất một vế đúng. Biểu thức bên trong ngoặc chạy trước phần `||`. Không cần biến mọi phép toán thành một câu đố — một tên như `urgent` thường dễ đọc hơn một điều kiện bị nén.

Function: đặt tên cho một quyết định có thể tái dùng

Khi điều kiện có ý nghĩa riêng, đưa nó vào hàm. Người đọc khi đó gặp một tên ở đúng mức trừu tượng thay vì phải giải mã lại cùng một điều kiện ở mọi nơi.

```
func classify(status int) string {  
    if status >= 500 {  
        return "failed"  
    }  
    return "healthy"  
}
```

Signature nói rõ hợp đồng tối thiểu: `classify` nhận một `int` và trả về một `string`. `status` là parameter, chỉ tồn tại trong lời gọi hàm. `return` không in gì cả; nó gửi một giá trị về nơi gọi. Vì thế câu `label := classify(status)` đọc theo thứ tự này: đọc `status` hiện có, gọi `classify` với giá trị đó, nhận `string` kết quả, rồi tạo `label`.

Cùng một quyết định, hai cách tổ chức

Nếu có nhiều trạng thái rời rạc, `switch` thường bộc lộ ý định tốt hơn một chuỗi `if`. Nó không “mạnh hơn”; nó chỉ đặt các nhánh cùng cấp cạnh nhau.

Đầu vào	Dạng phù hợp	Lý do đọc dễ hơn
Một ngưỡng: <code>status >= 500</code>	<code>if</code>	Điều kiện là một mệnh đề đúng/sai.
Vài giá trị cụ thể: 200, 404, 503	<code>switch</code>	Mỗi case là một lựa chọn cùng cấp.
Một thao tác lặp trên danh sách	<code>for</code> hoặc <code>range</code>	Ý định là làm lại, không phải chọn một nhánh.

```
func severity(status int) string {
    switch {
    case status >= 500:
        return "critical"
    case status >= 400:
        return "warning"
    default:
        return "normal"
    }
}
```

switch trên không có expression sau từ switch, nên từng case là bool expression. Go chạy case đầu tiên đúng rồi rời switch; không có fallthrough ngầm như một số ngôn ngữ khác. Thứ tự ở đây là dữ liệu của thiết kế: nếu đưa status >= 400 lên trước, 503 sẽ bị phân loại warning. Một compiler không thể bắt lỗi ý nghĩa đó cho bạn; đó là việc của người đọc và test.

Lặp là đi qua một tập giá trị

Một for có thể diễn tả vòng lặp điều kiện, nhưng range thường trực tiếp hơn khi ta muốn đi qua các phần tử của một collection.

```
statuses := []int{200, 503, 404}

for _, status := range statuses {
    fmt.Println(status, severity(status))
}
```

[]int là slice của int; cặp ngoặc nhọn tạo literal gồm ba phần tử. range cung cấp index và value. Tên _ bảo Go bỏ index đi vì ta không cần nó. Mỗi lượt, status là giá trị hiện tại; lời gọi severity dùng nó và Println in kết quả. Ta sẽ quay lại slice, bộ nhớ và aliasing ở chương sau — hiện tại chỉ cần thấy luồng đọc của vòng lặp.

Kiểu giới hạn những phép có nghĩa

Go suy ra nhiều kiểu từ literal, nhưng vẫn kiểm tra chúng chặt. `int` biểu diễn số nguyên; `bool` biểu diễn `true` hoặc `false`; `string` là một sequence of bytes không thể sửa trực tiếp. Text trong Go thường dùng UTF-8, và range trên `string` có semantics liên quan UTF-8/rune, nhưng `string` vẫn có thể chứa chuỗi byte không phải UTF-8 hợp lệ. Một biến có kiểu xác định; phép toán đòi các vế tương thích.

```
status := 503
message := "status: " + status // không hợp lệ
```

Toán tử `+` ở đây có thể nối hai `string` hoặc cộng hai số, nhưng không tự đoán cách trộn `string` và `int`. Chuyển đổi là một quyết định phải viết ra:

```
message := "status: " + fmt.Sprint(status)
```

`fmt.Sprint` tạo `string` để dùng cho hiển thị. Trong những ranh giới nghiêm túc hơn, ta sẽ phân biệt conversion với parsing và kiểm tra lỗi. Bây giờ điểm quan trọng là: kiểu giúp phát hiện một phép ghép vô nghĩa ngay tại nơi bạn viết nó.

Đọc compiler như một người cộng tác khó tính

Phần này là một cuộc điều tra nhỏ, không phải checklist lệnh. Trong lab `part1-reading-go`, giữ một bản đang chạy được. Sau đó tạo từng lỗi một, dự đoán trước, chạy compiler, và phân loại thông báo. Đừng sửa ngay khi vừa thấy chữ đỏ.

Case	Thay đổi có chủ ý	Dự đoán cần kiểm tra
A	Viết <code>status = 503</code> khi <code>status</code> chưa tồn tại	Lỗi thuộc về identifier hay giá trị?
B	Đổi <code>classify</code> nhận <code>string</code> nhưng caller vẫn đưa <code>status</code> kiểu <code>int</code>	Compiler nói về function hay type của argument?
C	Dùng <code>label</code> sau khi rời <code>if</code> block	Tên biến mất vì giá trị bị xóa hay vì scope kết thúc?

Kết quả dưới đây được chạy bằng Go 1.27.1 trên ba source file tối thiểu, mỗi case chỉ có một lỗi chủ ý. Dòng dài ở Case B được ngắt khi dàn trang; nội dung của compiler message được giữ nguyên.

```
[case-a]
# command-line-arguments
.\main.go:4:2: undefined: status

[case-b]
# command-line-arguments
.\main.go:9:15: cannot use status (variable of type int)
    as string value in argument to classify

[case-c]
# command-line-arguments
.\main.go:8:6: undefined: label
```

Case A là lỗi tên: compiler không thể gán cho một identifier chưa được khai báo. Case B đi xa hơn, vì tên tồn tại nhưng type của giá trị đưa vào không khớp parameter. Case C quay lại scope: `label` từng tồn tại trong block con, nhưng không thể được tham chiếu từ câu lệnh đứng sau block đó. Thông báo có thể khác đôi chút giữa phiên bản Go, còn ba loại lỗi thì vẫn là ba loại lỗi bạn cần nhìn ra.

Bài đọc cuối chương: không cần viết chương trình mới. Hãy nhìn hàm `severity`, dự đoán kết quả cho 200, 404 và 503, rồi viết một test table nhỏ cho ba dự đoán đó. Nếu test thất bại, lần theo branch được chọn thay vì sửa output một cách mù.

Đến đây, bạn đã có từ vựng để đọc phần lớn tệp Go nhỏ: package đưa tệp vào một đơn vị, declaration đặt tên, expression tạo giá trị, statement điều phối hiệu ứng, block giới hạn scope, và function biến một đoạn quyết định thành đơn vị có hợp đồng. Chương tiếp theo sẽ mở bằng một hiện tượng khó chịu hơn: hai biến trông độc lập, nhưng một thay đổi qua biến này lại hiện ra ở biến kia. Câu hỏi là chúng đã chia sẻ điều gì.

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu bạn đoán a và b khác nhau sau dòng `b[0] = 99`, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu “slice nguy hiểm”. Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy `b := a`, và dữ liệu nào không được copy?

Một baseline để dự đoán

Với một `int`, assignment tạo ra một giá trị độc lập. Cả a và b đều có giá trị riêng của chính chúng.

```
a := 10
b := a
b = 99

fmt.Println(a, b) // 10 99
```

Go đã copy value của a vào b. Sau đó b nhận value mới; a không có lý do gì để đổi. Đây là quy tắc nền: assignment copy value. Điều còn thiếu là một value có thể chứa gì.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên `[3]int` và `[4]int` là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```



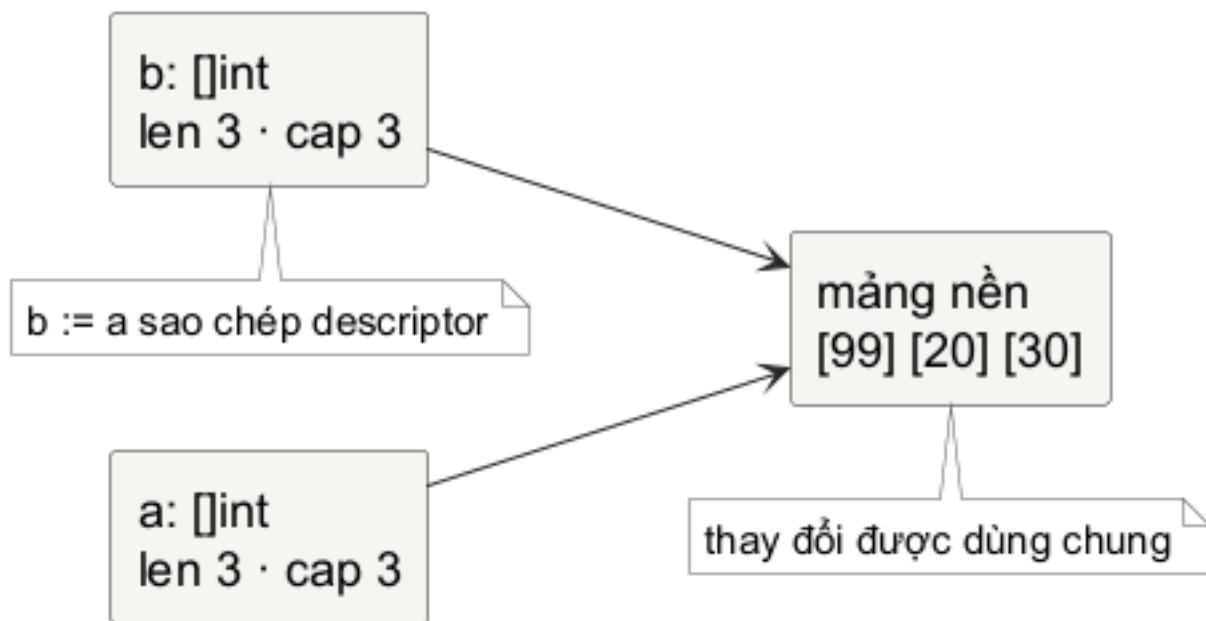
Hình 6 — Assignment của array copy các phần tử. Việc thay `b[0]` chỉ chạm vào array `b`.

Đến đây, trực giác “copy nghĩa là độc lập” vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: `[3]int` trở thành `[]int`.

Khoảnh khắc slice đổi kết quả

```
a := []int{10, 20, 30}
b := a
b[0] = 99
```

`[]int` không phải array không ghi độ dài. Nó là slice type. Một slice value mô tả một đoạn liên tiếp của underlying array. Ngôn ngữ định nghĩa slice value có length, capacity và một reference tới underlying array. Đó là mental model hữu ích cho semantics; nó không phải lời hứa rằng mọi Go implementation phải dùng đúng một struct hay ABI như hình.



Hình 7 — `b := a` copy slice value, gồm của số nhìn vào dữ liệu, len và cap. Cả hai descriptor vẫn dẫn tới cùng underlying array, nên `b[0]` thay đổi phần tử mà `a` cũng nhìn thấy.

Vì vậy hai câu sau cùng đúng, và không mâu thuẫn:

Câu nói	Đúng ở đâu?
Assignment copy value.	<code>b</code> nhận một slice value riêng: đổi len của <code>b</code> bằng <code>reslice</code> không tự đổi len của <code>a</code> .
Hai slice có thể cùng đổi dữ liệu.	Slice value có thể cùng reference một underlying array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Bảng 1 — Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

Cửa sổ, length và capacity

Đặt một slice lên một slice khác để thấy descriptor thực sự đang mô tả gì.

```
base := []int{10, 20, 30, 40, 50}
x := base[1:3]

fmt.Println(x)          // [20 30]
fmt.Println(len(x))     // 2
fmt.Println(cap(x))     // 4
```

x bắt đầu ở phần tử 20 của base và kết thúc trước 40. Vì thế length là 2. Capacity đếm số phần tử từ điểm bắt đầu của x đến cuối backing array mà x hiện có thể mở rộng tới; ở đây là 20, 30, 40, 50, nên là 4.



Hình 8 — x nhìn thấy hai phần tử đầu của cửa sổ. `cap(x)` không nói x đang có bốn phần tử; nó nói x có thể được reslice hoặc append trong phạm vi backing array hiện tại.

Hãy tự dự đoán trước khi chạy:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[1:3]
a[1] = 99

fmt.Println(base)
fmt.Println(a)
fmt.Println(b)
```

Kết quả là [1 99 3 4], [1 99], [99 3]. `a[1]` và `b[0]` đều chỉ vào phần tử `base[1]`. Hình vẽ không thay thế được việc lần index: nó chỉ cho biết vì sao có thể có một phần tử chung để lẫn.

append là một phép thử về capacity

append không hứa luôn dùng lại array cũ, cũng không hứa luôn tạo array mới. Nó trả về resulting slice. Khi backing array còn chỗ, implementation có thể dùng lại; khi capacity không đủ, append phải cấp underlying array mới đủ lớn cho kết quả. Vì return value mang descriptor có thể đã đổi, viết `s = append(s, value)` là dạng thông thường.

```
s := make([]int, 2, 4)
s[0], s[1] = 10, 20
old := s

s = append(s, 30)
s[0] = 99

fmt.Println(old) // [99 20]
fmt.Println(s)   // [99 20 30]
```

Ở đây `s` còn capacity. `old` và `s` có length khác nhau, nhưng vẫn có thể cùng nhìn hai element đầu của backing array. Ngược lại, full slice expression bên dưới giới hạn capacity của view ở 2, nên append buộc phải có storage khác cho grown:

```
old := []int{10, 20}
s := old[:2:2]
grown := append(s, 30)
grown[0] = 99

fmt.Println(old)    // [10 20]
fmt.Println(grown)  // [99 20 30]
```



Hình 9 — Hình mô tả hai semantics mà code phải chịu được. Đừng suy luận allocation từ một capacity tĩnh cở; nếu code cần một kết quả riêng, hãy tạo bản copy có chủ ý.

Một aliasing bug có người dùng thật

Một function “chỉ chuẩn bị preview” đôi khi phá dữ liệu của caller.

```
func redactPreview(fields []string) {  
    fields[0] = "***"  
}  
  
requestFields := []string{"api-key", "region"}  
redactPreview(requestFields)  
fmt.Println(requestFields) // [*** region]
```

Parameter `fields` nhận một slice value mới, nhưng descriptor ấy vẫn reference storage của `requestFields`. Việc đổi element là mutation shared storage. Nếu mục tiêu thật sự là tạo preview độc lập, copy phần tử sang một slice mới trước:

```
func redactedCopy(fields []string) []string {
    preview := make([]string, len(fields))
    copy(preview, fields)
    preview[0] = "***"
    return preview
}
```

copy sao chép tối đa số phần tử vừa với destination và trả về số phần tử đã copy. Ở đây make tạo backing array mới, nên preview độc lập ở từng slice elements. Đổi lấy independence là allocation và chi phí copy; với buffer lớn hoặc đường xử lý nóng, đó là một quyết định cần có lý do. Với một preview cần an toàn khỏi mutation, nó diễn đạt đúng intent.

Điều tra ngắn: hãy viết test cho redactedCopy. Test cần chứng minh đồng thời hai điều: kết quả có *** ở phần tử đầu, và requestFields ban đầu vẫn là api-key. Chỉ kiểm tra output preview chưa đủ để bắt aliasing bug.

String: bytes trước, text sau

String cũng là value, nhưng nó có luật riêng: string là immutable sequence of bytes. len đếm byte; index trả về byte. Text thường là UTF-8, song Go không yêu cầu mọi string phải chứa UTF-8 hợp lệ.

```
text := "Việt"
fmt.Println(len(text)) // 6

for index, value := range text {
    fmt.Println(index, value)
}
```

Lab chạy đoạn này với kết quả:

```
6
0 86
1 105
2 7879
5 116
```

V và i mỗi chiếm một byte. ệ được mã hóa bằng ba byte UTF-8, vì vậy rune tiếp theo bắt đầu ở byte index 5. range trên string decode UTF-8: giá trị thứ hai là rune, thường dùng để biểu diễn Unicode code point. byte là alias của uint8; rune là alias của int32. Code point vẫn không đồng nghĩa với “một ký tự người dùng nhìn thấy”: một grapheme có thể gồm nhiều code point. Ta chưa cần giải quyết segmentation ở đây; chỉ cần không nhầm lẫn với số ký tự hiển thị.

String không cho gán trực tiếp `text[0] = 'v'`. Khi cần dữ liệu byte có thể sửa, chuyển sang `[]byte`, thao tác, rồi tạo string mới nếu cần. Việc chuyển đổi đó diễn đạt một thay đổi ý định: từ text immutable sang buffer mutable.

Pointer là câu hỏi khác

Sau slice, pointer dễ bị giải thích sai như “slice nhưng rõ hơn”. Pointer là một value trực tiếp reference một variable khác; slice là một descriptor cho segment của underlying array. Chúng có thể cùng dẫn tới sharing, nhưng chúng trả lời hai câu hỏi khác nhau.

```
count := 10
p := &count
*p = 99
fmt.Println(count) // 99
```

Ở đây `&count` tạo pointer tới variable `count`, còn `*p` truy cập variable đó. Pointer sẽ có chương riêng khi ta cần thiết kế API quanh mutation trực tiếp và nil. Trước khi đến đó, hãy giữ model vừa xây: luôn hỏi value nào được copy, và value ấy có reference tới storage hay variable nào không.

Nếu câu trả lời là “không”, assignment thường tạo independence như int và array. Nếu câu trả lời là “có”, aliasing có thể là công cụ tốt hoặc một bug - tùy việc bạn có làm rõ ownership và mutation hay không.

Một quyết định ở ranh giới API

Khi một hàm nhận hoặc trả về slice, câu hỏi thiết kế không phải là “Go có copy không?” mà là: **ai được phép sửa storage này, và trong khoảng thời gian nào?** Một hàm lọc dữ liệu có thể trả về một view để tránh allocation; khi đó caller cần biết view ấy còn dùng chung buffer. Một hàm dựng response hoặc preview thường nên trả về dữ liệu độc lập, vì caller không có lý do để đoán một thay đổi sau đó sẽ đi ngược vào input.

Không có đáp án mặc định miễn phí. Copy tốn allocation và thời gian; chia sẻ tiết kiệm chúng nhưng làm contract về ownership quan trọng hơn. Trước khi tối ưu, hãy viết test cho hành vi mutation mà API hứa. Sau đó, khi đọc một bug khó hiểu, lần theo ba bước: value nào được gán, value đó reference storage nào, và ai còn có thể chạm storage ấy. Đó là cách biến “slice lạ quá” thành một cuộc điều tra có thể kiểm chứng.

Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi

opsprobe sẽ dần trở thành một CLI kiểm tra trạng thái dependency. Trước khi nó gọi HTTP hay đọc file config, nó đã có một bài toán nhỏ hơn: làm sao để biểu diễn trạng thái của một service mà người đọc không phải tự ghép bốn biến rời rạc trong đầu?

```
serviceName := "billing"
port := 8080
healthy := true
retries := 0
```

Bốn biến này đi cùng nhau. Chúng mô tả một service, nhưng compiler chưa biết điều đó. Càng nhiều hàm nhận chúng rời rạc, càng dễ đổi nhầm thứ tự, quên truyền một field, hoặc cập nhật một bản sao mà tưởng đã cập nhật trạng thái chung. Ta sẽ refactor dần mô hình này. Mỗi lần thay đổi, ta dừng ở cùng một câu hỏi: **function nhận value nào, và value ấy mở đường tới dữ liệu nào?**

Bug đầu tiên: bonus đi đâu mất?

Hãy tạm quên service. Một ví dụ nhỏ hơn cho thấy vấn đề rõ nhất:

```
balance := 100

func addBonus(x int) {
    x += 20
}

addBonus(balance)
fmt.Println(balance) // 100
```

balance không đổi vì lời gọi hàm cũng là một phép truyền value. Khi gọi `addBonus(balance)`, function nhận một parameter `x` có value 100. Câu `x += 20` thay value trong variable `x`; nó không quay lại gán cho variable

`balance` của caller. Đây không phải một ngoại lệ của `int`, mà là nền cho mọi signature trong chương này.



Hình 10 — Lỗi gọi `addBonus(balance)` copy value 100 vào parameter `x`. Sơ đồ biểu diễn semantics của variable và value, không khẳng định vị trí vật lý của chúng trong RAM.

Nếu function cần tạo một value mới, cách trên rất tốt: giá trị đi vào, kết quả đi ra bằng `return`. Ví dụ, `withBonus(balance)` có thể trả 120, và caller chọn có gán kết quả đó cho `balance` hay không. Nhưng một số thao tác có ý nghĩa là thay đổi chính variable caller đã đưa vào. Lúc ấy cần biểu đạt đường đi tới variable đó một cách rõ ràng.

Pointer: con đường tới một variable

Với operand addressable `x` có type `T`, `&x` tạo pointer value type `*T` trỏ tới variable `x`. Nếu `p` có type `*T`, `*p` là variable type `T` mà pointer đó trỏ tới. Hai câu này phân biệt năm thứ thường bị gộp làm một:

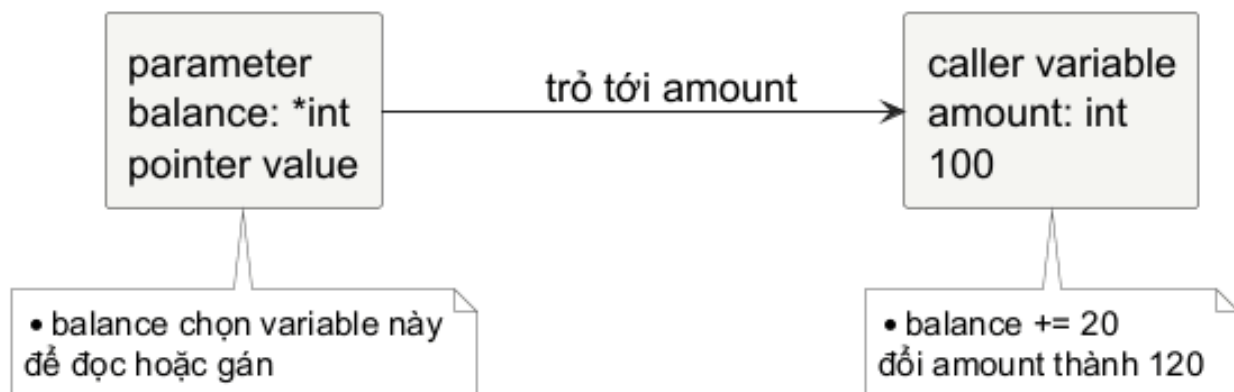
- **variable** là nơi chương trình có thể đọc hoặc gán value, như `balance`.
- **value** là dữ liệu đang được lưu, như 100.
- **address** là kết quả của phép lấy địa chỉ `&balance` trong ngữ cảnh Go.
- **pointer value** là value có type như `*int`; nó có thể trỏ tới một variable hoặc là `nil`.
- **pointee** là variable mà pointer hiện trỏ tới.

Ta không cần giả định variable ấy nằm cố định trên stack hay heap. Đó là quyết định implementation có thể thay đổi. Điều cần dùng khi đọc code là quan hệ ngôn ngữ: pointer value có đang trỏ tới một variable hợp lệ hay không.

```
func addBonus(balance *int) {
    *balance += 20
}

amount := 100
addBonus(&amount)
fmt.Println(amount) // 120
```

Ở caller, `&amount` tạo pointer tới variable `amount`. Parameter `balance` vẫn là một value được copy vào function - lần này value đó là pointer. Khi `*balance += 20` chạy, dereference `*balance` chọn chính variable `amount` làm đích gán. Bản sao của pointer không tạo bản sao của pointee.



Hình 11 — `balance` trong function là pointer value được truyền theo value. `*balance` truy cập pointee; sơ đồ cố ý không dùng địa chỉ số hay stack/heap để tránh biến mental model thành chi tiết runtime.

Phép `&` không dùng được với mọi expression. Go yêu cầu operand addressable: variable, dereference pointer, index của slice, field của struct addressable, index của array addressable, hoặc composite literal là những trường hợp quan trọng ở giai đoạn này. Map index không addressable - ta sẽ thấy lý do thực hành của giới hạn đó khi đưa map vào mô hình service.

Nil là thiếu pointee, không phải "pointer rỗng vô hại"

Zero value của pointer type là `nil`. `nil` không trở tới variable nào, nên dereference nó gây run-time panic.

```
var bonus *int

if bonus == nil {
    fmt.Println("chưa có bonus")
    return
}
*bonus += 20
```

Dereference `*bonus` cần một pointee để chọn làm đích gán; `nil` không có pointee, nên dereference nó panic. Kiểm tra `nil` là một phần của contract: function này có chấp nhận "không có giá trị" hay caller phải luôn truyền pointer hợp lệ? Đừng thêm `nil` check theo thói quen. Hãy quyết định và thể hiện điều đó qua API, test, hoặc lỗi trả về khi có lý do.

Gom state thành một value có tên

Quay lại bốn biến rời rạc. Một struct gom chúng thành một value mà domain có thể gọi tên:

```
type Service struct {
    Name      string
    Port      int
    Healthy    bool
    Retries    int
}

billing := Service{
    Name:      "billing",
    Port:      8080,
    Healthy:    true,
}
```

`type Service` khai báo một named type; phần thân struct khai báo các field và type của chúng. Struct literal đặt value cho các field được nêu. `Retries` không xuất hiện trong literal nên nhận zero value của `int`, là `0`. Zero value không thay thế validation: một `Service{}` hợp lệ về type nhưng chưa chắc hợp lệ cho một service thật. Trong domain này, port `0` có thể nghĩa là "chưa cấu hình".

Struct không phải class thiếu method. Trước hết nó là một value có nhiều phần mang cùng một ý nghĩa. Field access chỉ đọc hoặc gán một phần của value:

```
billing.Retries++
fmt.Println(billing.Name, billing.Retries) // billing 1
```

Một struct cũng được copy khi gán hoặc truyền làm argument. Với các field trong Service hiện tại, copy tạo các field độc lập:

```
candidate := billing
candidate.Healthy = false

fmt.Println(billing.Healthy) // true
fmt.Println(candidate.Healthy) // false
```



Hình 12 — Assignment copy struct value hiện tại theo từng field. Nếu một field về sau là slice, map hoặc pointer, struct vẫn được copy nhưng field đó có thể mang theo một reference; lúc ấy phải phân tích tiếp value của field thay vì gán nhãn "struct luôn độc lập".

Nested struct hữu ích khi một phần của domain tự nó đã có ý nghĩa. Ví dụ, host và port thường đi cùng nhau hơn là hai field phẳng:

```

type Endpoint struct {
    Host string
    Port int
}

type Service struct {
    Name      string
    Endpoint Endpoint
    Healthy   bool
    Retries   int
}

```

Việc nesting không tự động làm thiết kế tốt hơn. Nó có ích khi tạo ra một khái niệm đọc được, như `Endpoint`; gom chỉ để tránh thấy nhiều field thường làm model khó tìm hơn.

Khi function sửa một bản copy

Ta muốn `opsprobe` đánh dấu service không khỏe sau một lần probe thất bại. Phiên bản đầu tiên trông rất có lý:

```

func markUnhealthy(service Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(billing)
fmt.Println(billing.Healthy, billing.Retries) // true 0

```

Không có bug bí ẩn ở đây. `billing` được copy vào parameter `service`; function thay đổi struct copy rồi return. Caller còn giữ value cũ. Nếu đây là một transform thuần, ta có thể trả `Service` mới. Nếu ý định là mutate state caller đang quản lý, signature cần nói điều đó:

```
func markUnhealthy(service *Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(&billing)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

T và *T không có nghĩa "immutable" và "mutable". Một local value `Service` vẫn có thể bị gán field; một pointer parameter có thể chỉ được đọc. Khác biệt là một T parameter mang struct value riêng, còn một *T parameter mang khả năng truy cập pointee của caller. Chọn pointer khi function phải thay đổi pointee, khi biểu diễn sự vắng mặt bằng `nil` là một phần hữu ích của domain, hoặc khi API có lý do rõ ràng khác. Chọn value khi function cần snapshot cục bộ, transform rồi return, hoặc sharing sẽ làm contract khó hiểu hơn.

Cùng một lời gọi, bảy value khác nhau

Đến đây có thể đọc nhiều signature bằng một quy tắc duy nhất: Go truyền argument theo value. "Theo value" không trả lời xong câu hỏi; ta còn phải nhìn value đó chứa dữ liệu nào hoặc reference nào.

Parameter	Value được copy khi gọi	Thay đổi từ function có hiện ra ở caller?
<code>x int</code>	số nguyên	Không, trừ khi return và caller gán kết quả.
<code>a [3]int</code>	toàn bộ array value	Không, các element của bản copy bị sửa riêng.
<code>s []int</code>	slice value, gồm reference tới array nền	Sửa element có thể hiện ra; reslice parameter không tự đổi lên caller.
<code>m map[string]int</code>	map value, có reference tới map data	Ghi/xóa entry hiện ra qua map value cùng map data.
<code>p *int</code>	pointer value	<code>*p = ...</code> sửa pointee của caller nếu pointer hợp lệ.
<code>v Service</code>	các field của struct value	Không với các field value; xem từng field nếu chúng chứa reference.
<code>p *Service</code>	pointer value	Sửa field qua p sửa pointee của caller nếu pointer hợp lệ.

Bảng 2 — Mỗi lời gọi đều copy argument value vào parameter; khả năng chia sẻ nằm trong value được copy.

Bảng này không phải bảng để học thuộc type. Nó là cách lần một lời gọi: parameter mới nhận bản copy nào; bản copy ấy có dẫn đến dữ liệu chung hay variable chung không; expression nào là đích mutation? Đó cũng là lý do value receiver và pointer receiver sẽ được học sau cùng methods, thay vì biến dấu `*` thành một nghi thức.

Bài tập - sửa đúng contract: Viết `recordFailure(service Service)` sao cho caller nhận được `Healthy == false` và `Retries` tăng một. Trước khi nhìn đáp án, chọn một trong hai contract: function return một `Service` mới, hoặc function nhận `*Service` và mutate pointee.

Đáp án. Nếu caller quản lý cùng một state và function được đặt tên như một command, pointer làm ý định trực tiếp:

```
func recordFailure(service *Service) {
    service.Healthy = false
    service.Retries++
}

recordFailure(&billing)
```

Một biến thể value vẫn đúng khi API diễn đạt transform:

```
func failedCopy(service Service) Service {
    service.Healthy = false
    service.Retries++
    return service
}

billing = failedCopy(billing)
```

Hai phiên bản khác nhau ở trách nhiệm của caller. Phiên bản pointer cho phép function tác động lên state đã có; phiên bản return khiến việc thay state nằm rõ ở câu gán của caller. Không có "bản nhanh hơn" mặc định quan trọng hơn contract ở đây.

Nhiều service cần lookup, không cần thêm biến

Một Service model một service. Khi opsprobe giữ nhiều service theo tên, ta không muốn billing, search, checkout thành các variable độc lập. Map diễn đạt một quan hệ key/value:

```

services := map[string]Service{
    "billing": {
        Name:    "billing",
        Port:    8080,
        Healthy: true,
    },
}

billing, found := services["billing"]
fmt.Println(found, billing.Port) // true 8080

```

string là key type; Service là element type. found là kết quả thứ hai của map lookup, thường gọi là comma-ok. Nó phân biệt key không có mặt với element có zero value. Điều này quan trọng với counter: services["missing"].Retries không phải cách hợp lệ để đọc field, nhưng với map[string]int, lookup key thiếu và key có value 0 đều cho 0 nếu chỉ nhận một result.

```

attempts := map[string]int{"billing": 0}

count, found := attempts["billing"]
fmt.Println(count, found) // 0 true

count, found = attempts["search"]
fmt.Println(count, found) // 0 false

```

Map literal tạo map đã sẵn sàng ghi. make(map[string]Service) cũng tạo map sẵn sàng ghi. Ngược lại, zero value của map là nil: đọc từ nil map cho zero value, range qua nó có zero iteration, nhưng gán entry vào nil map sẽ panic. Một map nhận từ caller vì thế cần contract rõ như pointer: caller có luôn khởi tạo nó không, hay function chịu trách nhiệm tạo nó trước khi ghi?

```
var services map[string]Service
fmt.Println(services["billing"].Name) // ""

// services["billing"] = billing // panic: assignment to entry in nil map
services = make(map[string]Service)
services["billing"] = billing
```

`delete(services, "billing")` xóa một entry; xóa key không có mặt là an toàn. Nhưng `range` map không có thứ tự được chỉ định. Nếu CLI cần output ổn định, hãy lấy key, sort chúng, rồi in theo thứ tự đã chọn - đừng coi thứ tự đang tình cờ xuất hiện là contract của map.

Map copy value, chia sẻ map data

Map không phải pointer và ta không cần mô tả hash table internals để dùng nó đúng. Assignment và call copy map value. Nếu hai map value cùng reference map data, ghi qua một value hiện ra qua value kia:

```
registry := map[string]int{"billing": 1}
alias := registry
alias["billing"]++

fmt.Println(registry["billing"]) // 2
```



Hình 13 — Assignment copy map value. Khi value ấy reference cùng map data, mutation entry qua `alias` hiện ra khi đọc qua `registry`; hình là conceptual diagram, không mô tả hash table implementation.

Một chi tiết làm map khác slice và array khi update struct value: map index không addressable. Vì vậy đoạn này không hợp lệ:

```
// services["billing"].Healthy = false // không hợp lệ
```

`services["billing"]` cho một `Service` value để đọc; nó không cho một `variable` mà ta có thể lấy địa chỉ field hay gán field trực tiếp. Cách `value-oriented` là lấy value ra, đổi, rồi gán lại:

```
service := services["billing"]
service.Healthy = false
services["billing"] = service
```

Nếu domain muốn registry quản lý các `*Service`, thì mutation qua pointee có thể trực tiếp hơn, nhưng đổi lại registry có thể chứa nil pointer và ownership cần được làm rõ:

```
services := map[string]*Service{
    "billing": &Service{Name: "billing", Port: 8080, Healthy: true},
}

services["billing"].Healthy = false
```

Đây không phải lời mời chọn `map[string]*Service` mặc định. Với value nhỏ như `Service` hiện tại, `map[string]Service` và "lấy - sửa - gán lại" thường bộc lộ điểm mutation rõ hơn. Pointer trong map chỉ đáng dùng khi nó giải quyết một contract thực: object identity dùng chung, mutation phải đi qua nhiều nơi, hoặc value lớn và đã có lý do đo đạc/thiết kế.

Bài tập - đọc zero value: Một probe trả về `latencyByService map[string]int`. Viết code phân biệt `billing` có latency 0 với `billing` chưa có kết quả. Sau đó xóa kết quả của `billing` mà không cần kiểm tra key có tồn tại.

Đáp án. Comma-ok mang thông tin hiện diện; `delete` tự xử lý key thiếu:

```
latency, found := latencyByService["billing"]
if !found {
    fmt.Println("chưa có kết quả")
} else {
    fmt.Println(latency)
}

delete(latencyByService, "billing")
```

Một update có trách nhiệm trong registry

`opsprobe` chưa cần network để hưởng lợi từ model này. Một update value-oriented cho registry không cần pointer: lấy struct value ra khỏi map, sửa bản local, rồi gán nó trở lại entry. Map parameter được truyền theo value, nhưng map value ấy vẫn mở đường tới map data của caller; chính phép gán cuối là mutation có chủ ý lên registry chung.

```

func recordProbe(registry map[string]Service, name string, healthy bool)
    service, found := registry[name]
    if !found {
        return false
    }

    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
    registry[name] = service
    return true
}

```

Signature này nói rõ ba vai: `registry` là map value để lookup và gán entry; `name` là snapshot string; `healthy` là snapshot bool. `service` là struct copy lấy từ entry. Nếu key không có, function không bịa ra service mới mà trả `false`; nếu có, nó ghi value đã sửa về map. Khi công cụ bắt đầu đọc config và probe endpoint ở phần sau, cách đọc signature này sẽ giúp ta biết dữ liệu nào được chia sẻ, dữ liệu nào được snapshot, và mutation nào có thể lặp lại.

Sau chương này, một signature không còn chỉ là tên type. Nó cho biết ta đang đưa vào function một snapshot, một cửa sổ tới storage dùng chung, một map value dẫn tới map data, hay một pointer mở đường tới variable caller. Khi cần debug mutation, hãy viết ra trước/sau lời gọi: variable nào tồn tại ở caller, parameter nào xuất hiện ở function, và expression nào thực sự là đích gán.

Khi hành vi thuộc về service

`recordProbe(registry, name, healthy)` đã làm đúng việc, nhưng khi `opsprobe` có thêm nhiều thao tác quanh một service, chữ ký bắt đầu lặp lại cùng một chủ thể. Một hàm cần tạo nhãn cho service, một hàm khác ghi kết quả probe, hàm nữa kiểm tra cấu hình. Lúc này method giúp đặt hành vi cạnh type mà nó đọc hoặc thay đổi; nó không thay đổi quy tắc value semantics đã xây.

```
func (service Service) Summary() string {
    return service.Name
}

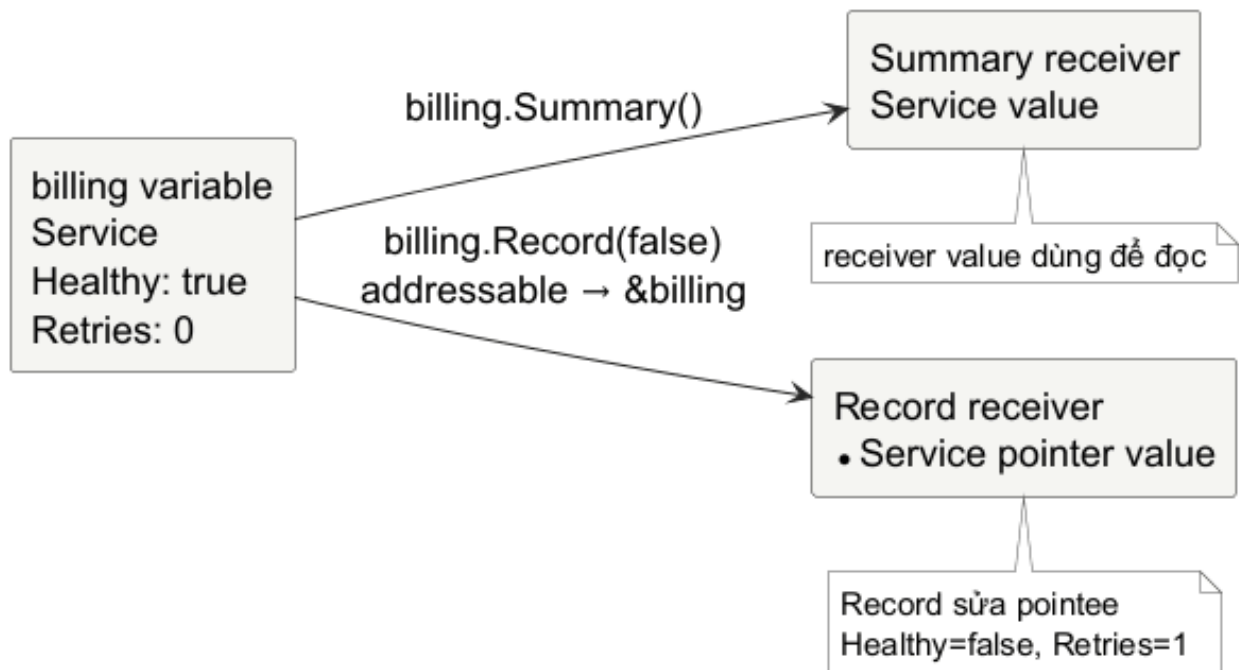
func (service *Service) Record(healthy bool) {
    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
}
```

Một method là function có receiver: phần parameter đặc biệt đứng trước tên method. `Summary` nhận `Service` value, nên receiver là một struct value được dùng để tạo kết quả. `Record` nhận `*Service`, vì hành vi này có ý định sửa pointee. Tên receiver không phải từ khóa và không phải một biến toàn cục bí mật; nó chỉ là parameter được viết ở vị trí riêng để gắn method với `Service`.

```
billing := Service{Name: "billing", Healthy: true}

fmt.Println(billing.Summary()) // billing
billing.Record(false)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

`billing.Record(false)` hợp lệ vì `billing` là variable addressable. Trong trường hợp này, Go cho phép cách viết ngắn của `(&billing).Record(false)`. Sự tiện lợi ấy không biến value receiver thành pointer receiver: method set của `Service` chỉ có methods khai báo với receiver `Service`; method set của `*Service` có cả methods receiver `Service` lẫn `*Service`.



Hình 14 — Method vẫn dựa trên receiver value. Summary dùng Service value; Record cần *Service để chọn pointee làm đích mutation. Mũi tên là trace semantics, không mô tả bộ nhớ vật lý.

Điều này giải thích một lỗi compiler rất có ích. Map index không addressable, nên Go không thể tự lấy địa chỉ ổn định cho một value trả về từ map để gọi pointer-receiver method:

```
// services["billing"].Record(false) // không hợp lệ

service := services["billing"]
service.Record(false)
services["billing"] = service
```

Đoạn sửa dùng đúng mô hình cũ: map lookup cho struct value, local variable `service` addressable, pointer-receiver method mutate local pointee, rồi caller gán struct value đã đổi về map. Nếu code thật cần nhiều nơi cùng giữ identity của một service, `map[string]*Service` có thể hợp lý; nhưng không dùng pointer chỉ để né phép gán lại mà chưa hiểu ownership.

Composition trước, embedding sau

Một service có endpoint và state; hai phần đó là các khái niệm riêng. Composition đơn giản nhất là đặt chúng vào những field có tên, để người đọc thấy quan hệ "có một":

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeState struct {
    Healthy bool
    Retries int
}

type Service struct {
    Name      string
    Endpoint  Endpoint
    State     ProbeState
}
```

Service có Endpoint và ProbeState; không có quan hệ kế thừa nào được ngụ ý. Đây là cách composition thường rõ nhất khi domain có các phần độc lập. Embedding cũng là một công cụ của Go, nhưng nó đưa field và method được promote vào selector của type ngoài. Ta sẽ chỉ dùng nó khi một API có lợi từ sự promote đó; hiện tại field có tên giữ ownership và đường đi của data rõ hơn.

Interface xuất hiện ở nơi cần một hành vi

Một interface không phải bản thiết kế cha để mọi struct đi theo. Nó là một contract nhỏ: code tiêu dùng nói mình cần method nào, còn type nào có method set phù hợp thì dùng được. Với opspoke, phần in kết quả chưa cần toàn bộ Service; nó chỉ cần một dòng summary.

```

type SummarySource interface {
    Summary() string
}

func renderSummary(source SummarySource) string {
    return "target: " + source.Summary()
}

```

renderSummary không hỏi caller là Service, StaticTarget, hay một type tương lai. Nó chỉ gọi Summary. Không có từ khóa implements: assignment hoặc argument hợp lệ khi method set của type thỏa interface. Vì Summary có value receiver, cả Service và *Service đều có method đó trong method set.

```

type StaticTarget string

func (target StaticTarget) Summary() string {
    return string(target)
}

fmt.Println(renderSummary(Service{Name: "billing"}))
fmt.Println(renderSummary(StaticTarget("search (maintenance)")))

```

Kết quả là hai dòng có cùng format dù hai type không có ancestor chung. Interface ở đây được rút ra sau khi có consumer và hai provider có ý nghĩa. Nếu chỉ có một Service duy nhất và không có ranh giới cần thay thế, nhận trực tiếp Service thường dễ đọc hơn là thêm interface vì dự đoán tương lai.



Hình 15 — `renderSummary` chỉ cần `Summary() string`. Sơ đồ cho thấy dependency đi từ consumer tới contract nhỏ; `Service` và `StaticTarget` không cần khai báo một quan hệ kế thừa hay đăng ký implements.

Pointer receiver thay đổi interface satisfaction theo đúng rule method set. Nếu contract yêu cầu `Record(bool)`, `Service` value không thỏa vì method đó chỉ thuộc method set của `*Service`; `*Service` mới thỏa.

```

type ProbeRecorder interface {
    Record(bool)
}

func markFailed(recorder ProbeRecorder) {
    recorder.Record(false)
}

billing := Service{Name: "billing", Healthy: true}
markFailed(&billing) // *Service thỏa ProbeRecorder

// markFailed(billing) // sai: Service thiếu Record
  
```

Bài tập - chọn contract nhỏ: `renderSummary` cần biết `Name`, `Port`, `Healthy` và `Retries` hay chỉ cần `Summary() string`? Viết một type thứ hai có thể đi qua `renderSummary` mà không phải là `Service`, rồi giải thích vì sao interface đặt ở consumer thay vì được nhét sẵn vào mọi type.

Đáp án. Consumer chỉ cần `Summary() string`, nên interface nên giữ đúng một method đó. `StaticTarget` ở ví dụ trên là một provider thứ hai: nó có thể là dữ liệu đọc từ một report thay vì service sống. Đặt interface cạnh `renderSummary` khiến contract phản ánh nhu cầu thật của consumer; `Service` không phải biết trước mọi interface mà code khác sẽ cần.

Một API nhỏ, đủ để đọc bằng semantics

Ở cuối mạch này, `opsprobe` có thể diễn đạt ba ý định khác nhau mà không cần dựa vào danh từ mơ hồ như "reference type":

- `func (service Service) Summary() string` nhận snapshot receiver và tạo mô tả.
- `func (service *Service) Record(bool)` nhận pointer value, rồi sửa pointee có chủ ý.
- `func renderSummary(SummarySource) string` nhận interface value và chỉ phụ thuộc vào hành vi mà consumer cần.

Composition giữ data model có đường đi rõ ràng; methods đặt behavior cạnh type; interface tạo ranh giới khi có một consumer thực sự cần thay thế provider. Từ đây, chương về errors có thể cho những API này một cách báo thất bại rõ ràng: lỗi probe không nên chỉ đổi `Healthy` rồi biến mất, mà phải đi qua ranh giới function với thông tin đủ để caller quyết định.

Chương 4 — Biên lỗi: để caller quyết định

opsprobe vừa có thêm Service, state và một hàm ghi kết quả probe.

Phiên bản đầu của hàm trả bool: true nếu tên service tồn tại, false nếu không. Nó là một API hợp lệ khi caller chỉ cần biết có ghi được entry hay không.

Nhưng hãy đặt nó vào một tình huống vận hành. Màn hình báo billing không healthy. Có ít nhất hai câu hỏi khác nhau: service có được cấu hình không, hay service đã cấu hình nhưng probe tới endpoint thất bại? Một false không mang câu trả lời nào đi qua ranh giới function. Caller buộc phải đoán, hoặc đọc state đã bị đổi sau sự kiện.

Chương này không bắt đầu bằng một danh sách hàm trong package errors. Nó bắt đầu bằng quyền quyết định: nơi gọi cần nhận được failure nào để chọn thông báo, retry, hay dừng công việc; nơi phát hiện failure cần thêm context nào mà không phá mất identity của nguyên nhân.

Một failure chưa phải là một quyết định

Ta tách hành vi probe thành một function value. Nó chưa mở network; điều đó sẽ đến ở chương I/O. Hiện tại nó là một boundary có thể thành công hoặc trả về nguyên nhân thất bại:

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeFunc func(Endpoint) error
```

ProbeFunc nhận endpoint value và trả về error. Nếu không có lỗi, result là nil. Nếu có lỗi, caller nhận một error value. error là interface của Go; điều đáng chú ý ở đây không phải interface đó có một method, mà là failure đi ra cùng result channel với một shape thống nhất. Không dùng panic để báo endpoint unreachable, và cũng không in log bên trong ProbeFunc rồi giả vờ thành công.

API ghi kết quả của `opsprobe` đổi từ câu hỏi “có entry không?” sang “tôi đã probe và cập nhật state được chưa?”

```
func applyProbe(  
    registry map[string]Service,  
    name string,  
    run ProbeFunc,  
) error
```

Signature nói một điều quan trọng trước khi ta đọc implementation: caller không nhận state mới hay một boolean mơ hồ. Nó nhận `nil` khi operation hoàn tất, hoặc một error value để quyết định nhánh kế tiếp. `registry` vẫn là map value dẫn tới shared map data; vì map đang lưu struct value, hàm vẫn lấy entry ra, sửa local `Service`, rồi gán lại entry như ở Chương 3.

Phân loại để quyết định, thêm context để điều tra

Không phải mọi string lỗi đều là contract. Một CLI có thể muốn đối xử riêng với “tên service không có trong config”, nhưng không nên parse câu tiếng Anh từ `err.Error()` để làm điều đó. Ta tạo một sentinel cho điều kiện mà caller có hành động riêng:

```
var ErrUnknownService = errors.New("service is not configured")
```

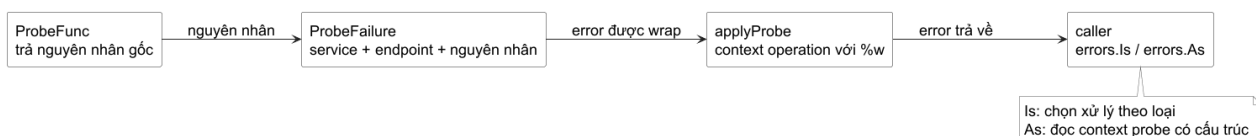
Khi tên không tồn tại, `applyProbe` không trả sentinel trần. Nó thêm thông tin operation và vẫn giữ nguyên identity bằng `%w`:

```
return fmt.Errorf("unknown %q: %w", name, ErrUnknownService)
```

`%w` không chỉ nối string. Nó tạo một error có thể unwrap về nguyên nhân. Vì vậy caller có thể kiểm tra ý nghĩa bằng `errors.Is` thay vì so câu chữ:

```
if errors.Is(err, ErrUnknownService) {
    fmt.Println("thêm service vào config trước khi probe")
    return
}
```

Trong câu lệnh trên, caller không cần biết hiện có bao nhiêu lớp context bên ngoài sentinel. `errors.Is` đi theo chuỗi wrapping. Nếu đổi `%w` thành `%v`, thông báo có thể nhìn giống nhau nhưng identity bị cắt; một điều kiện có thể phân loại được đã vô tình thành text chỉ dành cho người đọc.



Hình 16 — Biên lỗi không nuốt nguyên nhân. Mỗi lớp chỉ thêm context mà nó thực sự biết; caller chọn `errors.Is` khi cần một quyết định theo loại lỗi và `errors.As` khi cần dữ liệu từ một error cụ thể.

Một error có dữ liệu mà CLI thực sự cần

Sentinel trả lời “đây có phải loại lỗi đó không?”. Khi probe tới một endpoint đã có cấu hình nhưng thất bại, CLI cần thêm service nào và endpoint nào liên quan. Ta dùng custom error cho context có cấu trúc:

```

type ProbeFailure struct {
    Service  string
    Endpoint Endpoint
    Cause    error
}

func (failure *ProbeFailure) Error() string {
    return fmt.Sprintf(
        "probe %s at %s:%d: %v",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
        failure.Cause,
    )
}

func (failure *ProbeFailure) Unwrap() error {
    return failure.Cause
}

```

Error tạo message cho người đọc. Unwrap nói rõ Cause vẫn là nguyên nhân nằm dưới lớp context này. ProbeFailure dùng pointer receiver; vì vậy khi ta cần lấy nó từ error chain, target của errors.As là *ProbeFailure:

```

var failure *ProbeFailure
if errors.As(err, &failure) {
    fmt.Printf("%s đang lỗi tại %s:%d\n",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
    )
}

```

errors.As không hỏi “message có chứa host không?”. Nó tìm một error assignable vào type mà caller yêu cầu. Đây là lý do custom error nên có dữ liệu mà caller thật sự dùng, không phải chỉ là một struct bọc mỗi string.

Failure analysis: state nào phải được ghi, lỗi nào phải được trả?

Đây là implementation của boundary. Hãy đọc nó như một review: mỗi return mang thông tin nào, và state nào còn tồn tại sau return?

```
func applyProbe(
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error {
    service, found := registry[name]
    if !found {
        return fmt.Errorf("unknown %q: %w", name, ErrUnknownService)
    }

    if err := run(service.Endpoint); err != nil {
        service.Record(false)
        registry[name] = service
        failed := &ProbeFailure{
            Service:  service.Name,
            Endpoint: service.Endpoint,
            Cause:    err,
        }
        return fmt.Errorf("probe %q: %w", name, failed)
    }

    service.Record(true)
    registry[name] = service
    return nil
}
```

Nhánh thiếu config trả sớm: không có Service để mutate, nên registry không đổi. Nhánh probe thất bại vẫn ghi state `Healthy=false` và tăng retry, rồi trả failure có nguyên nhân. Đây không phải hai cơ chế báo lỗi cạnh tranh; state trả lời “lần quan sát gần nhất ra sao”, còn error trả lời “operation vừa gọi không hoàn tất vì sao”. Nhánh thành công ghi state rồi trả `nil`.

Khi kiểm thử code này, có ba contract cần chứng minh thay vì chỉ kiểm tra message:

- Với service thiếu, `errors.Is(err, ErrUnknownService)` là true và probe function không được gọi.
- Với probe thất bại, `errors.As` lấy được `*ProbeFailure`, `errors.Is` vẫn thấy nguyên nhân gốc, và state đã được ghi lại.
- Với probe thành công, error là nil và endpoint truyền vào probe đúng với config.

Lab part4-error-boundaries biến ba contract này thành test.

ProbeFunc được thay bằng function nhỏ trong test; không cần mở socket thật để chứng minh error boundary.

Bài tập - giữ lại điều gì? Một run trả `errors.New("connection refused")`. Nếu `applyProbe` đổi `%w` ở return cuối thành `%v`, message của CLI có thể vẫn hiển thị “connection refused”. Contract nào trong ba contract trên đã mất? Viết một assertion thể hiện điều đó.

Đáp án. Contract nhận biết nguyên nhân gốc mất: `errors.Is(err, errConnectionRefused)` sẽ false vì chuỗi error không còn unwrap tới error ban đầu. Context cho người đọc không đủ để thay thế identity cho code. Assertion cần tạo `errConnectionRefused`, gọi `applyProbe`, rồi yêu cầu `errors.Is(got, errConnectionRefused)` là true.

Điểm dừng của boundary này

Ta chưa retry, chưa timeout, và chưa cleanup tài nguyên; mọi thứ đó sẽ chỉ làm mạch này khó đọc nếu đưa vào trước. Hiện tại API đã có ranh giới rõ: nil nghĩa là operation hoàn tất, error giữ được nguyên nhân và context, còn caller dùng Is hoặc As đúng theo quyết định mình cần.

Phần tiếp theo của Chương 4 sẽ đặt error boundary này dưới áp lực của tài nguyên và cancellation: defer phải chạy ở đâu, context đi qua API nào, và vì sao panic không phải đường tắt thay cho một failure có thể dự đoán.

Cancellation không phải health signal

Một probe có thể bị hủy vì người dùng dừng CLI, vì request chưa hết thời gian, hoặc vì service quản lý gọi đang shutdown. Không tình huống nào trong số đó chứng minh endpoint của billing đang down. Nếu `applyProbe` gộp mọi error vào nhánh `Record(false)`, một cancellation của caller sẽ tạo ra một health signal giả.

`context.Context` mang deadline và tín hiệu cancellation qua ranh giới API. Nó đi vào đầu signature, không được cất trong `Service`, và được truyền xuống function có thể chặn:

```
type ProbeFunc func(context.Context, Endpoint) error

func applyProbe(
    ctx context.Context,
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error
```

Trước khi chạy probe, boundary kiểm tra cancellation để không bắt đầu I/O vô ích. Nếu cancellation đến trong lúc `run` đang làm việc, `run` phải tôn trọng `ctx` và trả `ctx.Err()`. `applyProbe` nhận diện hai nguyên nhân chuẩn `context.Canceled` và `context.DeadlineExceeded`, giữ chúng trong error chain, nhưng không thay health state:

```

if err := ctx.Err(); err != nil {
    return fmt.Errorf("probe canceled: %w", err)
}

if err := run(ctx, service.Endpoint); err != nil {
    if errors.Is(err, context.Canceled) ||
        errors.Is(err, context.DeadlineExceeded) {
        return fmt.Errorf("probe incomplete: %w", err)
    }

    service.Record(false)
    registry[name] = service
    failed := &ProbeFailure{
        Service: service.Name,
        Endpoint: service.Endpoint,
        Cause: err,
    }
    return fmt.Errorf("probe %q: %w", name, failed)
}

```

Đây là một policy của opspoke, không phải luật rằng cancellation không bao giờ có ý nghĩa vận hành. Tại boundary này, state `Healthy` có nghĩa là kết quả của lần quan sát hoàn tất. Một operation bị hủy chưa đưa ra quan sát đó. Caller vẫn có thể dùng `errors.Is(err, context.Canceled)` để dừng yên lặng, hoặc `errors.Is(err, context.DeadlineExceeded)` để báo timeout.

defer đứng cạnh acquisition

Cancellation không tự đóng mọi resource mà một function đã mở.

Ownership nằm ở nơi acquisition thành công: function nào có session thì function đó phải sắp cleanup trước khi gọi phần có thể return. Ta tách phần này khỏi health state để đọc được policy cleanup mà không lẫn nó với policy endpoint:

```

type ProbeSession interface {
    Close() error
}

type OpenSession func(Endpoint) (ProbeSession, error)
type SessionProbe func(
    context.Context,
    Endpoint,
    ProbeSession,
) error

func runWithSession(
    ctx context.Context,
    endpoint Endpoint,
    open OpenSession,
    run SessionProbe,
) (err error) {
    if err := ctx.Err(); err != nil {
        return err
    }

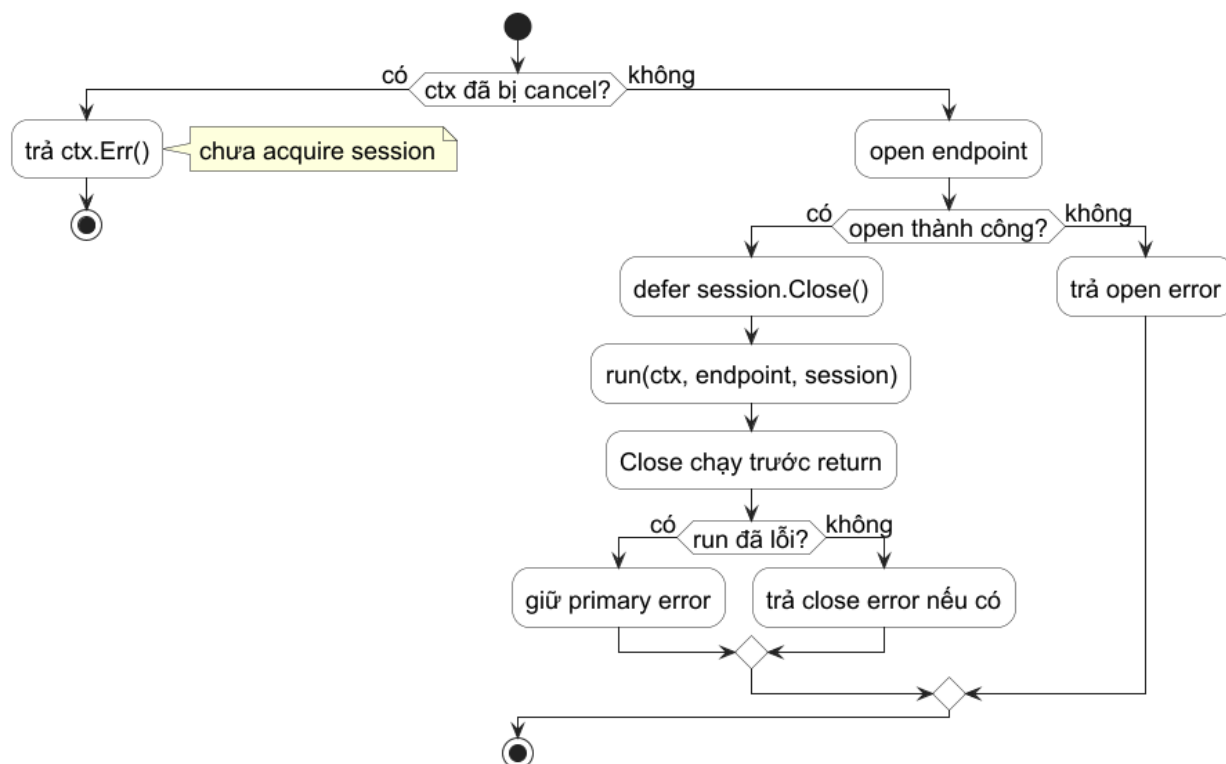
    session, err := open(endpoint)
    if err != nil {
        return fmt.Errorf("open session: %w", err)
    }
    defer func() {
        if closeErr := session.Close(); closeErr != nil {
            if err == nil {
                err = fmt.Errorf("close session: %w", closeErr)
            }
        }
    }()

    return run(ctx, endpoint, session)
}

```

defer được đăng ký ngay sau khi open thành công. Nếu open thất bại, chưa có session để đóng. Nếu run return theo bất kỳ nhánh nào, Close chạy trước khi runWithSession trả result cuối cùng. Named result err ở đây có lý do cụ thể: cleanup error chỉ trở thành result khi operation chính đã thành công. Nếu run đã có error, code giữ primary failure thay vì thay thế

nó bằng một lỗi close thứ cấp. Một hệ thống có yêu cầu lưu cả hai có thể dùng policy khác, nhưng phải làm điều đó có chủ ý.



Hình 17 — **defer** không phải một câu thần chú dọn dẹp toàn chương trình. Nó ràng buộc cleanup với scope đã nhận ownership. Error từ **Close** chỉ thay result khi **run** chưa có failure để bảo toàn nguyên nhân chính.

Lưu ý một ranh giới khác: caller tạo context dẫn xuất phải gọi **cancel**, kể cả khi deadline có thể tự hết. Trong code ngắn, pattern là `ctx, cancel := context.WithTimeout(parent, timeout)` ngay sau đó `defer cancel()`. `cancel` giải phóng resource liên quan tới context; còn `session.Close` giải phóng resource mà `runWithSession` đã acquire. Hai cleanup này có owner khác nhau.

Code review: đừng dùng **recover** để che failure dự đoán được

Một review có thể gặp đoạn sau và thấy nó “tránh crash”:

```
defer func() {  
    if recovered := recover(); recovered != nil {  
        err = fmt.Errorf("probe failed: %v", recovered)  
    }  
}()
```

Đoạn này không phải cách thay thế cho `return error`. Cancellation, timeout, config thiếu và connection refused đều là outcome mà API đã dự kiến; chúng phải đi qua error chain để caller phân loại được. Dùng `recover` ở đây biến bug lập trình như nil dereference hoặc invariant hỏng thành một string mơ hồ, trong khi state có thể đã bị thay đổi dở dang.

`panic` dành cho tình trạng không thể tiếp tục theo contract nội bộ hoặc cho những boundary được thiết kế riêng để cách ly code không tin cậy. `recover` chỉ có chỗ khi boundary đó có kế hoạch phục hồi state, ghi nhận đầy đủ và có contract rõ ràng sau recovery. `opsprobe` hiện không có boundary như vậy; để panic lộ ra trong test thường trung thực hơn là giả nó thành “probe failed”.

Bài tập - trace cleanup: `ctx` bị cancel sau khi open trả một session, còn `run` trả `context.Canceled`. `Close` có được gọi không? Error cuối giữ identity nào? `registry["billing"]` có nên đổi `Healthy` hay `Retries` không?

Đáp án. `Close` được gọi đúng một lần vì `defer` đã được đăng ký sau acquisition. `RunWithSession` giữ `context.Canceled` là primary error nếu `Close` cũng lỗi; `applyProbe` bọc nguyên nhân đó để `errors.Is(err, context.Canceled)` vẫn true. Vì operation không hoàn tất một quan sát endpoint, policy hiện tại giữ nguyên `Healthy` và `Retries`.

Kết chương: failure có đường đi

Một error boundary tốt không cố làm mọi failure giống nhau.

`ErrUnknownService` cho caller một loại lỗi để cấu hình; `ProbeFailure` giữ context của quan sát thật; wrapping giữ identity; cancellation đi qua `Context` nhưng không bị viết nhầm thành health signal; `defer` đóng resource mà scope đã nhận ownership. `panic` vẫn là tín hiệu khác, không phải lỗi tắt để khởi thiết kế result.

`opsprobe` nay nói rõ điều nó sở hữu, quan sát và để caller quyết định. Tiếp theo: package boundary, export và dependency kín.

Bốn câu hỏi trước khi merge một error boundary

- Caller có một quyết định khác nhau cho từng outcome không? Nếu có, hãy giữ identity bằng sentinel hoặc type có dữ liệu; đừng bắt caller parse message.
- Failure này có phải quan sát về domain không? Timeout hay cancellation của caller không tự động là bằng chứng endpoint unhealthy.
- Resource được acquire ở đâu, và cleanup có được đăng ký ngay sau acquisition không? Mỗi `defer` nên chỉ rõ scope nào đang nhận ownership.
- Nếu cleanup cũng lỗi, primary failure nào không được phép biến mất? Policy có thể khác giữa hệ thống, nhưng cần được kiểm thử như một contract.

Khi bốn câu trả lời hiện ngay trong signature, error chain, state transition và test, người đọc sau không cần đoán error có bị nuốt, retry có bị tăng sai, hay resource có bị bỏ quên. Đó là chuẩn bị cần thiết trước khi tách code thành package.

Chương 5 — Package là ranh giới

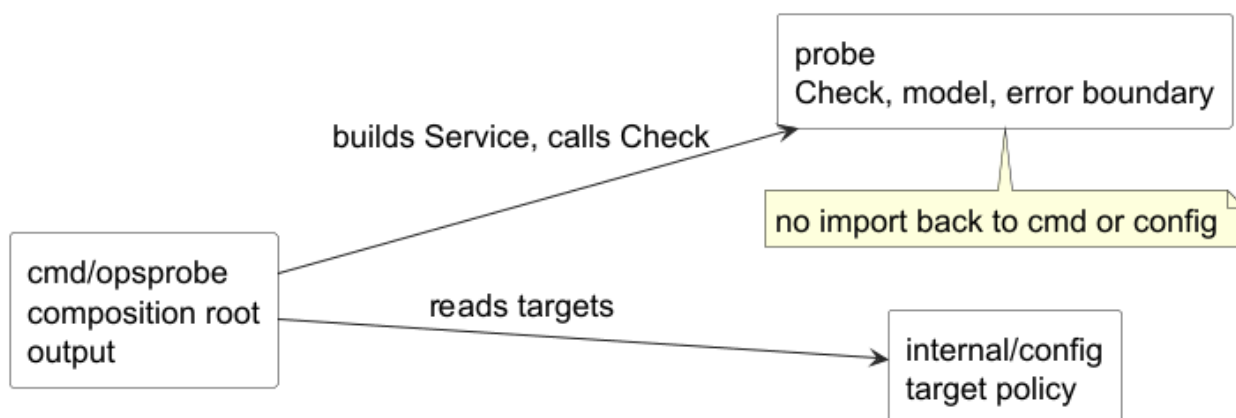
Sau Chương 4, `opspoke` có một API cho một lần probe. Nhưng lab vẫn là một package `main` chứa `model`, `policy error`, `config` giả và cách in ra terminal. Khi mọi thứ còn ngắn, một tệp là lựa chọn tốt. Vấn đề xuất hiện không phải vì số tệp tăng, mà vì một thay đổi ở cách in CLI có thể nhìn thấy mọi chi tiết của probe, và một thay đổi config có thể kéo domain model đi cùng.

Ta sẽ không mở đầu bằng cây thư mục “chuẩn”. Ta bắt đầu bằng một review: code nào phải biết `fmt.Println`, code nào phải biết endpoint, và code nào có quyền quyết định config được đọc từ đâu? Khi câu trả lời khác nhau, boundary cũng phải khác nhau.

Một dependency graph có thể đọc từ ngoài vào trong

Refactor đầu tiên của `opspoke` có ba vai:

- `cmd/opspoke` là command: nhận các dependency, gọi use case và trình bày kết quả.
- `probe` là package importable: sở hữu model cho một lần kiểm tra và không in ra terminal.
- `internal/config` là implementation detail của application: biết target mặc định hôm nay, nhưng không trở thành API công khai mà module khác được hứa hỗ trợ.



Hình 18 — Dependency direction là công cụ review. `cmd/opspoke` là composition root; nó biết các package cụ thể. `probe` không biết CLI hay config source nên có thể được đọc, test và thay thế runner mà không kéo presentation theo.

`go.mod` đặt module path, còn mỗi directory Go là một package. Vì lab dùng module `example.com/golang-master/part5-package-design`, import path của package probe là `example.com/golang-master/part5-package-design/probe`. Một package có thể gồm nhiều file trong cùng directory; chia file không tạo thêm boundary nếu chúng vẫn khai báo cùng package probe.

```
part5-package-design/
├── go.mod
├── cmd/opsprobe/main.go
├── probe/doc.go
├── probe/check.go
└── internal/config/config.go
```

Tên `cmd` là convention hữu ích khi repository có thể chứa nhiều executable; bản thân compiler không trao ý nghĩa đặc biệt cho nó. `main.go` cũng chỉ là tên file theo convention. Boundary thực sự đến từ package clause, import path và hướng dependency.

Export là lời hứa, không phải cách sửa compile error

Trong probe, caller cần tạo service, truyền một runner và đọc result. Những khái niệm ấy mới cần export:

```
package probe

type Endpoint struct { Host string; Port int }
type Service struct { Name string; Endpoint Endpoint }

type Runner func(context.Context, Endpoint) error
type Result struct { Service string; Healthy bool }

func Check(
    ctx context.Context,
    service Service,
    run Runner,
) (Result, error)
```

Chữ cái đầu viết hoa không làm type “quan trọng hơn”; nó mở một contract cho package khác. Check được export vì command thực sự gọi nó. Helper thêm context error ở bên trong có thể giữ unexported vì chưa có caller nào ngoài probe cần nó. Export trước rồi hy vọng một consumer xuất hiện là cách tạo API phải mang gánh nặng tương thích không cần thiết.

Package documentation cũng là một phần của contract. File `probe/doc.go` bắt đầu bằng comment `Package probe . . .`, nói package chịu trách nhiệm gì và không chịu trách nhiệm gì. Comment của export nên mô tả behavior mà caller có thể dựa vào, không chỉ lặp lại tên type.

`internal` là boundary do compiler kiểm tra

Target mặc định là policy của application hiện tại. Ta để nó trong `internal/config`:

```
package config

type Target struct {
    Name string
    Host string
    Port int
}

func DefaultTargets() []Target {
    return []Target{
        {Name: "billing", Host: "billing.internal", Port: 8443},
    }
}
```

Package ở `internal` không phải “private bằng thỏa thuận”. Go chỉ cho code nằm trong cây thư mục của parent import nó. Vì `cmd/opsprobe` nằm dưới module root - parent của `internal` - nó import được `internal/config`. Một module khác import `example.com/golang-master/part5-package-design/internal/config` sẽ bị go command từ chối.

Đây không phải lý do để nhét mọi package vào `internal` theo phản xạ. Với binary tự chứa, đó thường là default tốt vì không hứa API ngoài. Với probe

trong lab này, ta cố tình để nó importable để boundary giữa domain operation và CLI được nhìn thấy. Nếu mai kia không còn consumer ngoài application, nó cũng có thể trở thành internal; vị trí phải theo consumer thật, không theo cây thư mục đẹp mắt.

Composition root chuyển dữ liệu, không giấu dependency

`config.Target` và `probe.Service` không phải cùng type. Chúng thuộc hai boundary khác nhau: config nói dữ liệu được khai báo thế nào; probe nói một operation cần gì. `cmd/opsprobe` là nơi phù hợp để map giữa chúng:

```
for _, target := range config.DefaultTargets() {
    service := probe.Service{
        Name: target.Name,
        Endpoint: probe.Endpoint{
            Host: target.Host,
            Port: target.Port,
        },
    },
}

result, err := probe.Check(ctx, service, run)
if err != nil {
    fmt.Printf("%s: %v\n", service.Name, err)
    continue
}
fmt.Printf(
    "%s healthy=%t\n",
    result.Service,
    result.Healthy,
)
}
```

Mapping này trông như boilerplate nhỏ, nhưng nó giữ dependency direction trung thực: probe không cần import config, config không cần biết Result, và code in terminal không rò vào domain package. Khi source config chuyển từ default sang file hay environment ở chương sau, `probe.Check` không đổi chỉ vì đường đi của config đã đổi.

Một refactor có kiểm chứng, không phải một lần dời file

Lab kiểm tra ba điều có thể bị hỏng khi tách package:

- `probe.Check` chuyển cancellation thành error có thể nhận diện bằng `errors.Is`, và không chạy runner khi context đã bị cancel.
- Result thành công mang đúng service name và `Healthy=true`.
- `internal/config` trả dữ liệu độc lập theo từng lần gọi; caller sửa slice nhận được không làm thay default của lần sau.

Test cuối không phải để khẳng định “mọi slice đều dangerous”. Nó chứng minh ownership của config factory: `DefaultTargets` tạo dữ liệu mới cho caller thay vì phát một slice dùng chung. Đây là cùng câu hỏi value semantics của Chương 2, nay được đặt vào một package boundary.

Bài tập - đặt code ở đâu? Một bạn muốn thêm `fmt.Println` vào `probe.Check` để in progress, và muốn `probe` gọi `config.DefaultTargets` để đỡ mapping ở `main`. Với dependency graph trên, hai thay đổi đó tạo ra rủi ro gì? Đề xuất nơi thay thế cho mỗi việc.

Đáp án. `fmt.Println` kéo presentation vào package có thể import; command nên quyết định format và nơi ghi output. Để `probe` gọi config đảo dependency từ domain operation sang application policy, khiến test `probe` phải mang config theo. Progress có thể là result/event do caller trình bày; mapping nên ở command hoặc một application service nằm phía ngoài cả `probe` lẫn `config`.

Điểm dừng: package đủ nhỏ để thay đổi

Chưa có một “architecture hoàn chỉnh” nào ở đây: chưa parse file, chưa dùng generic, chưa tạo package cho từng noun. Ta chỉ có một graph mà mỗi mũi tên trả lời được “vì sao package này cần biết package kia?”. Đó là mức cấu trúc đủ để tiếp tục thay đổi code mà không biến mọi thay đổi thành sửa `main`.

Phần tiếp theo của Chương 5 sẽ dùng boundary này để đọc configuration thật và document API; generics chỉ xuất hiện khi một quan hệ type lặp lại được chứng minh, không phải vì module mới có nhiều folder.